Latent Diffusion Models

Nathan Van Maastricht

March 20, 2026

Diffusion

Diffusion

Generative diffusion is the process of learning to reverse entropy

We literally take random noise and turn it into coherent data

Physical Diffusion

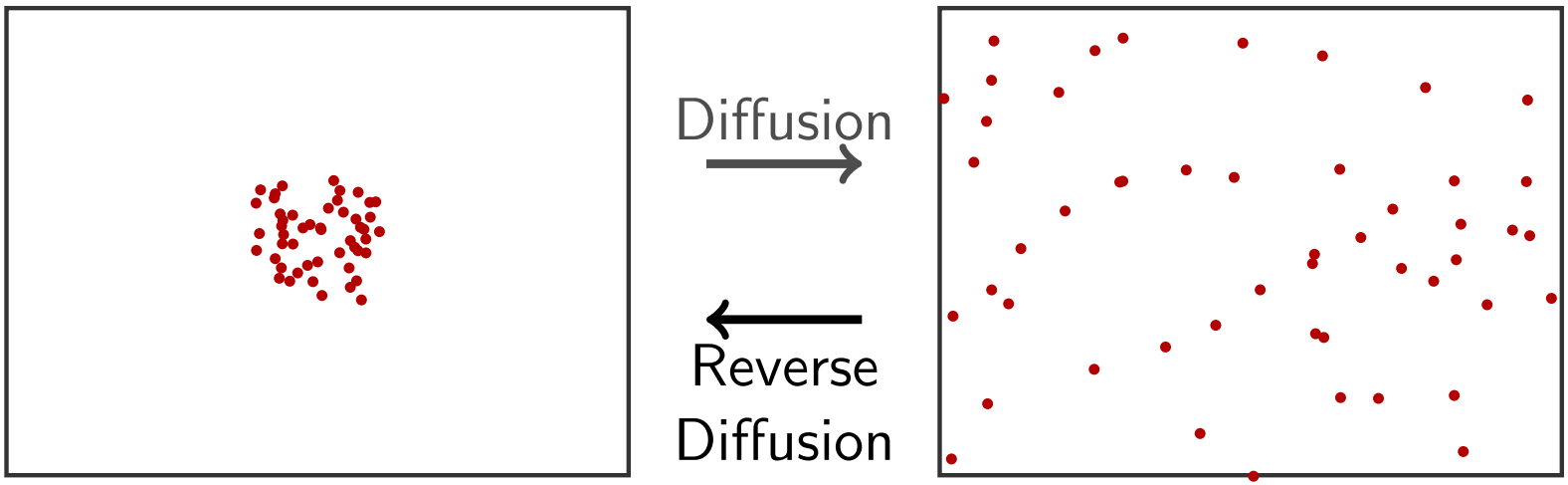
Mechanism

Spontaneous movement of particles from regions of high concentration to low concentration

Generative Approach

If we can precisely model the local drift that causes this dispersion, we can mathematically invert time to recover structure from the chaos

Physical Diffusion



Flow Matching

Define a time dependent vector field $v_t(x)$ that describes the velocity of a smple moving from a simple distribution (such as Gaussian nosie) to the complex data distribution (such as real images). This is encoded in the relatively basic looking ODE

$$\frac{dx}{dt} = v_t(x)$$

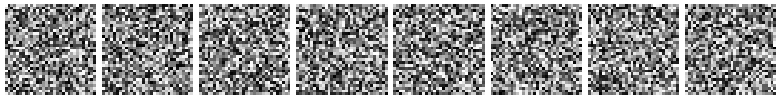
Learning Objective

We learn this velocity field directly.

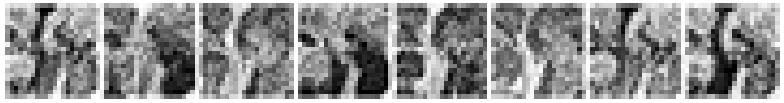
Random Noise
$\epsilon \sim \mathcal{N}(0, I)$

Sampling

Latent Space Diffusion
Flow Matching / ODE Solving



Latent Vector (z)



VAE Decoder
$p(x|z)$



Final Image (x)

Data

- CC12M is a dataset composed by google, with roughly 6M image/caption pairs still available
- Re-LAION-Caption 19M is by supermodel research derived from LAION 5B, using roughly 3M images, with four potential captions each, focusing on subject, setting, aesthetic, camera details

The VAE's Job

1. Encode images to latents
2. Decode latents to images
3. Structure the latent space so a diffusion model can learn how it is structured

VAE Architecture

- Residual Blocks
- Attention Blocks
- Upsampling Blocks
- Downsampling Blocks
- FSQ Layer

Residual Blocks

- Preactivation style (normalise and activation first)
- Additive residuals (add rather than concatenate)

```
h = self.conv1(self.act(self.norm1(x)))
h = self.conv2(self.act(self.norm2(h)))

return h + self.shortcut(x)
```

Attention Blocks

- Global Context
- Flash Attention 2 for fast attention kernels

```
qkv = self.qkv(self.norm(x))
q, k, v = torch.chunk(qkv, 3, dim=1)

out = F.scaled_dot_product_attention(
    q, k, v,
    dropout_p=self.dropout,
    scale=self.scale
)

return x + self.proj_out(out)
```

Spatial Resizing

- **Downsampling:** Uses a convolutional layer with a 3×3 kernel and a stride of 2 to reduce the spatial dimensions by half
- **Upsampling:** Uses nearest neighbour interpolation to upscale the spatial dimensions by a factor of 2, followed by a convolutional layer to reduce checkerboard artifacting that appears with transposed convolutions

The downsampling block and upsampling block are attempting to be mirror images of each other with respect to the spatial dimensions

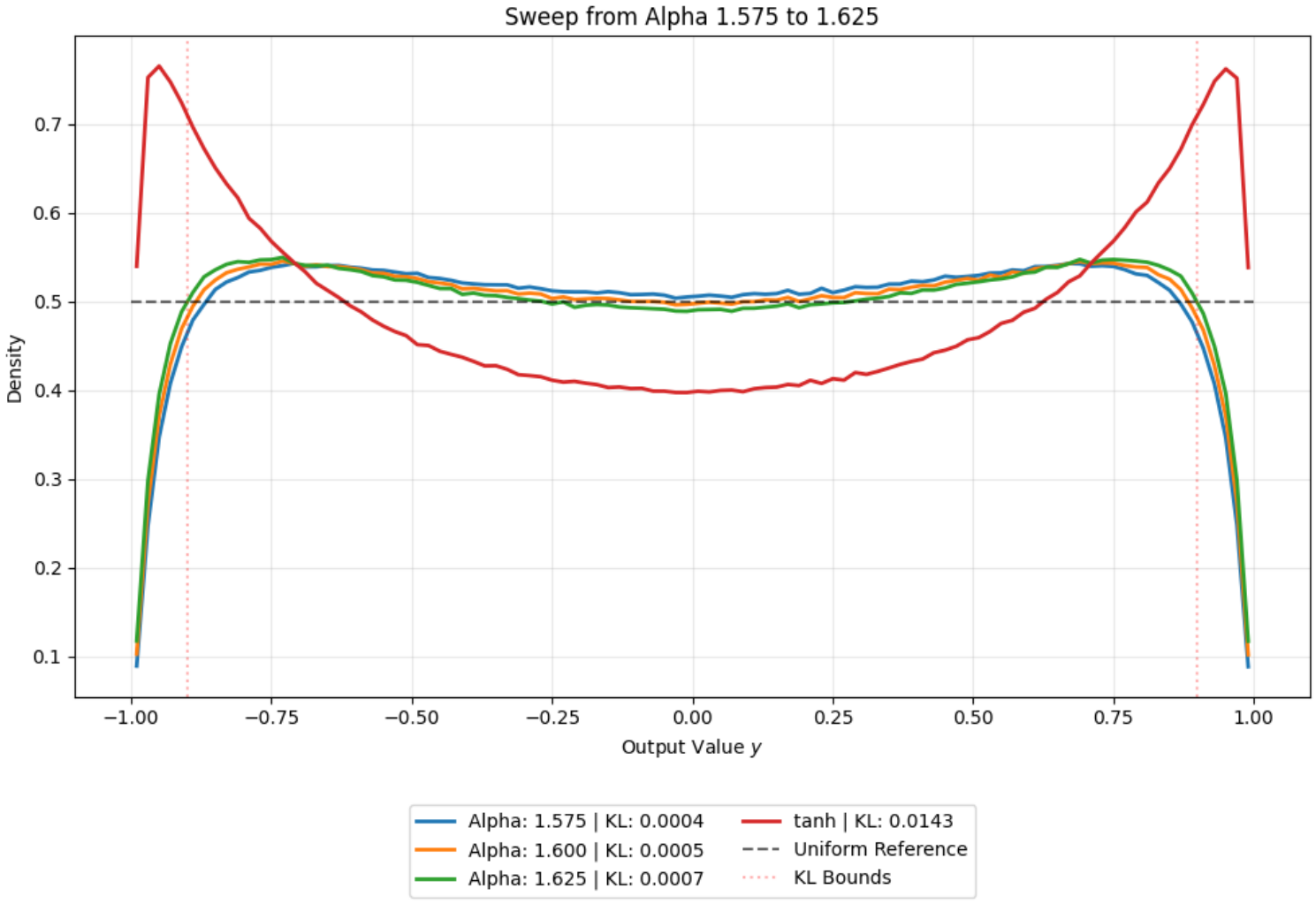
FSQ Layer

- Map continuous latents to a discrete grid
- Use a scaled sigmoid $(2\sigma(1.6x) - 1)$ to squash values into $[-1, 1]$
- Round values to fixed levels, in my case 15 levels per dimension
- Use a straight through estimator to bypass the non-differentiable rounding function

Why FSQ-VAE?

- No codebook collapse: Standard VQ-VAE have dead entries that never get used because the codebook is learned. In an FSQ-VAE the levels are predefined
- The straight through estimator passes the correct quantised latent to the decoder, and the correct gradients back to the encoder, while going through a non differentiable function. Very similar to the reparameterisation trick in a KL-VAE

Why $2\sigma(1.6x) - 1$?



VAE Summary

- Model type: FSQ-VAE
- Input Resolution: $256 \times 256 \times 3$
- Quantised Grid: $[15, 15, 15, 15, 15, 15, 15, 15]$
- Codebook size: 15^8
- Downsampling Factor: $8 \times (256 \to 32)$
- Compression Factor: $24\times$. $256 \times 256 \times 3$ $(196,608) \to 32 \times 32 \times 8$ $(8,192)$

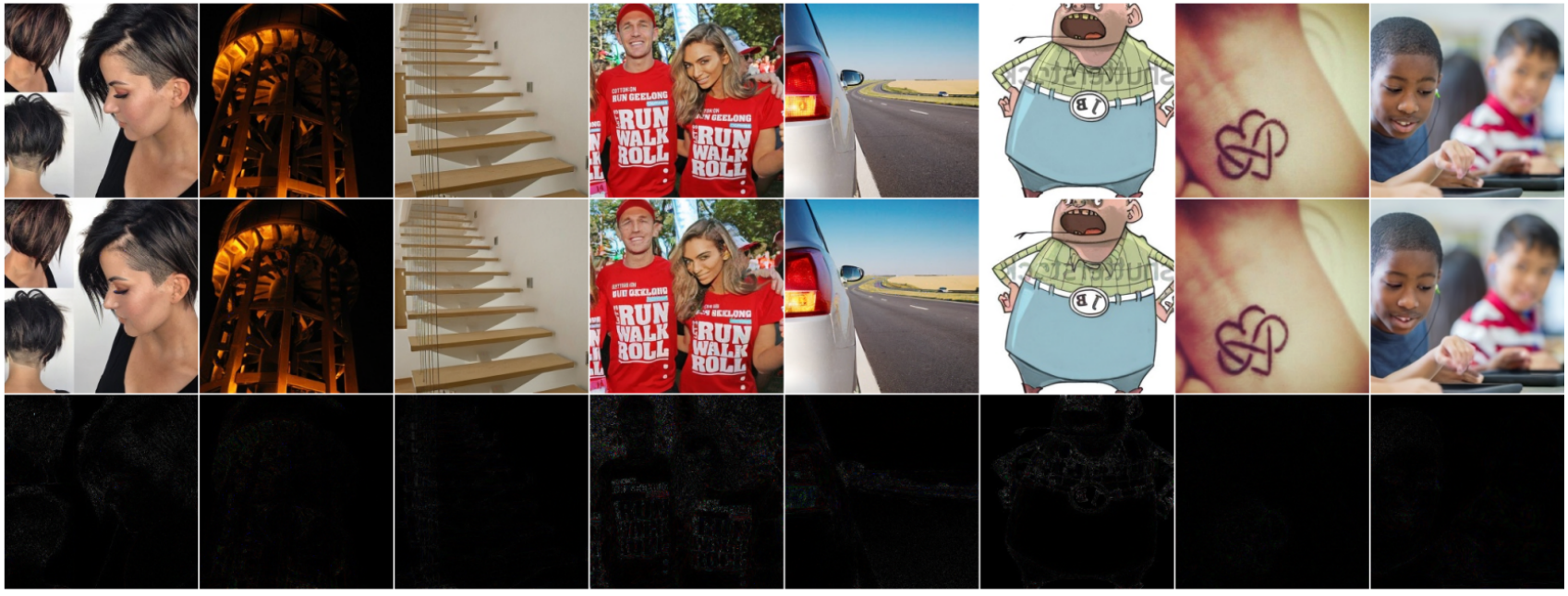
Training the VAE: Loss

- Reconstruction: L1 measures pixel-wise fidelity, ensuring the global structure is correct. Often leads to blurriness if the only component of the loss.
- Adversarial: A discriminator model is fed the original image and reconstructed image. The VAE wants to fool the discriminator, which pushes it to produce sharper images.
- Feature Matching: Using the same discriminator model, compare the internal activations for the real and fake images to ensure statistical and perceptual features match.

Training the VAE: Warmup and Cooldown

- Adversarial Warmup: The adversarial component of the loss function linearly grows to a maximum value, allowing the VAE to partially learn reconstruction before being pushed by the discriminator to shape the latent space appropriately. This is roughly 10% of the training time
- Learning Rate: The learning rate starts at roughly 10% of it's maximum rate, and linearly grows over the first 10% of training. Then it is held at this maximum rate until the last 10% of it's training, where it decays along a cosine curve, allowing the model to fall deep into the local minimum it has found

VAE Training Visualisations



VAE: What's Been Skipped?

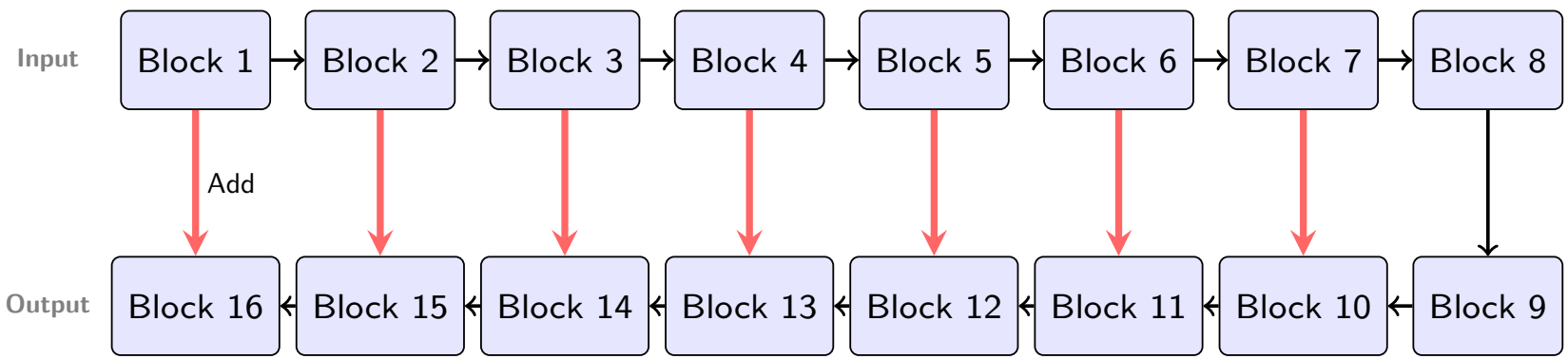
- Code book utilisation monitoring
- Program optimisation such as memory management, device management
- Training compromises such as gradient accumulation, what effective batch size is useful, how hyperparameter values were selected, why some values had warmup and some didn't
- Any details on the discriminator at all, for example why use spectral normalisation
- Gradient clipping

Training DiT-UNet

- The VAE encodes images into latents
- Noise is added to the latents proportional to a time variable between 0 and 1
- The DiT-UNet predicts a proxy for this noise called “velocity”

Architecture

- 16 Transformer blocks using patch-based attention
- Skip connections link early blocks to late blocks
- Self-attention to capture spatial dependencies
- Cross-attention to inject text conditioning


Input
Block 1
Block 2
Block 3
Block 4
Block 5
Block 6
Block 7
Block 8
Add
Output
Block 16
Block 15
Block 14
Block 13
Block 12
Block 11
Block 10
Block 9

Attention

- Latent tokens (patches) are related to each other by self-attention, with 2D RoPE for relative spatial relationships
- Mapping text tokens to spatial regions is done with cross-attention

Adaptive Layer Norm

Instead of a static Layer Norm γ, β, and α are calculated per transformer block from the combined time and text embedding.

These are used in a gating mechanism, allowing less or more weight for the transformers text parameters, initially not even using the transformers parameters related to text embeddings.

Preprocessing

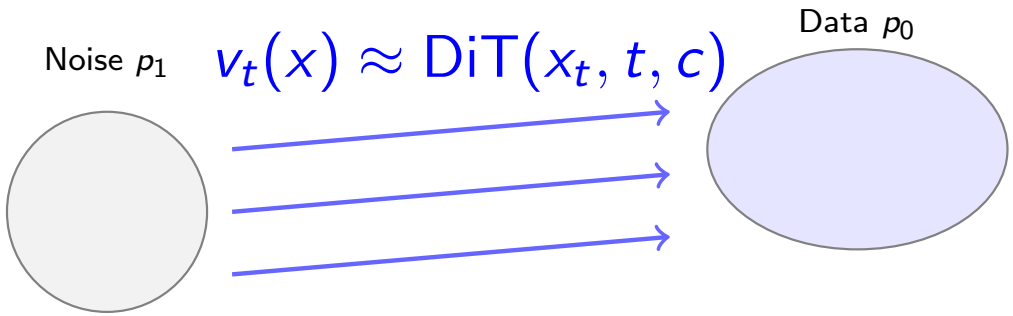
- Encode all images and captions ahead of time
- Ensure dataloader has enough bandwidth on storage to keep the compute saturated

Training the DiT-UNet

To recap, we're trying to train a model to learn the vector field $v_t(x)$ that satisfies

$$\frac{dx}{dt} = v_t(x)$$

which corresponds to learning a straight line path from the noise p_1 to the data p_0.



$$\mathcal{L}_{FM} = \mathbb{E}_{t,x_0,x_1} \left\| v_t(x_t) - (x_1 - x_0) \right\|^2$$

Optimisations and Stability

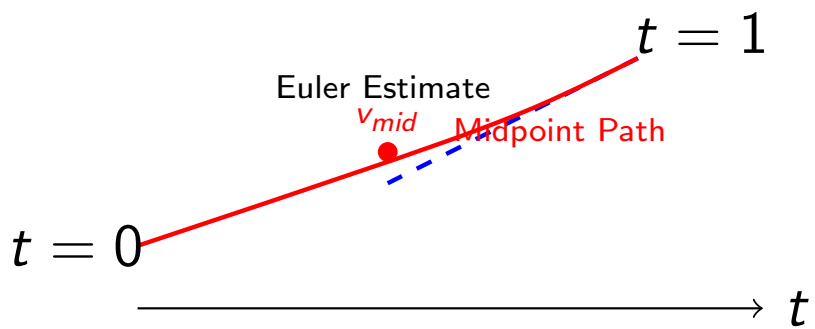
- EMA smooths out the weight updates
- Training in bf16 is sufficient for most calculations, with the loss being calculated in full precision still
- Learning rate warmup, hold, and cooldown
- Classifier free guidance

DiT: What's been skipped?

- How to implement the attention mechanism, in particular how to normalise the QKV tensors
- Why we only apply RoPE to the self-attention but not the cross-attention
- Why this differential equation?
- Why predict the proxy rather than noise directly?
- Issues with data handling
- What's being monitored
- Gradient clipping

We've got a model, now what? RK2

1. Take a half-step using the initial velocity predicted by the model
2. Re-evaluate the velocity field at that midpoint
3. Take the full step using the refined velocity



Classifier-Free Guidance

Classifier-Free Guidance moves the sample away from the "generic" distribution and toward the specific "text-aligned" distribution, attempting to focus the model.

There is also the concept of a negative prompt, pushing away from concepts such as "blurred" or "out of focus".

Post Sampling

- Unnormalise from DiT-UNet to VAE decoder
- FSQ Snapping: Round the output of the DiT to the level the FSQ-VAE expect

Sample



What's been skipped?

- Signals to watch for throughout training?
- How to train the language model that is being used to inject the prompts?
- How to determine what a "generic" distribution is?
- How to normalise and unnormalise the data at the various stages of training
- Guard rails
- How long should training go for?
- How to determine any of this the hyperparameters?
- References? Where did I learn this?
- Alternatives? What do the big companies do that I don't have the resources for? How do the big companies differ to each other? Who even thought this was reasonable to try?
- Beside pictures, where can this be used?

Mid-training



Magic?

